BugBoard26

Documento di design del sistema

Progetto di Ingegneria del Software
A.A. 2025/2026

Componenti	Valerio Porcile, Emanuel Giuseppe Lupoli, Emanuele Martucci
Modalità	Tradizionale
Versione	1.1
Data	9 ottobre 2026

Indice

1 Introduzione **2**

2 Obiettivi di progettazione **2**

3 Architettura generale **3**
3.1 Component diagram . . . 3
3.2 Deployment diagram . . . 4

4 Scelte tecnologiche **6**
4.1 Frontend Angular . . . 6
4.2 Backend ASP.NET Core Web API . . . 6
4.3 Database PostgreSQL . . . 6
4.4 Autenticazione JWT . . . 7

5 Organizzazione del backend **7**
5.1 Controller . . . 7
5.2 Servizi applicativi . . . 7
5.3 DbContext . . . 7

6 Class diagram di design **8**
6.1 Vista dominio e persistenza . . . 8
6.2 Vista controller e dipendenze . . . 9

7 Schema della persistenza dati **10**

8 Principali scelte di design **11**
8.1 Separazione frontend e backend . . . 11
8.2 Uso di servizi applicativi . . . 11
8.3 Accesso ai dati . . . 11
8.4 Gestione dei ruoli . . . 11
8.5 Suggerimento automatico dello assegnatario . . . 12
8.6 Gestione delle notifiche . . . 13
8.7 Gestione dei duplicati . . . 14
8.8 Archiviazione . . . 14
8.9 Consultazione ed export CSV . . . 14
8.10 Immagine opzionale della issue . . . 14

9 Diagrammi comportamentali **14**
9.1 Creazione di una issue . . . 15
9.2 Cambio stato con notifica . . . 16
9.3 Contrassegno di una issue duplicata . . . 17

10 API principali implementate **18**

11 Testabilità **19**
11.1 Analisi qualità backend . . . 19

12 Limiti e verifiche aperte **19**

13 Conclusioni **20**

1 Introduzione

Il presente documento descrive la progettazione del sistema BugBoard26, realizzato come applicazione distribuita per la gestione collaborativa di issue in progetti software.

Il documento rappresenta la seconda fase del lavoro progettuale e riprende quanto definito nella specifica dei requisiti. In particolare, vengono descritte le scelte architetturali, la suddivisione delle responsabilità tra frontend, backend e database, la struttura principale delle classi di design e le motivazioni delle principali scelte tecnologiche.

Le funzionalità considerate sono quelle assegnate al gruppo:

- autenticazione tramite email e password;
- creazione di utenti da parte degli amministratori;
- creazione di issue da parte di ADMIN e USER autenticati, con immagine opzionale caricata separatamente;
- visualizzazione, ricerca, filtro e ordinamento delle issue;
- cambio stato delle issue assegnate;
- notifica al segnalatore quando una issue viene risolta;
- esportazione delle issue in formato CSV;
- archiviazione delle issue da parte degli amministratori;
- suggerimento automatico dello assegnatario in base al carico di lavoro;
- gestione di utenti readonly;
- contrassegno di una issue come duplicata.

2 Obiettivi di progettazione

La progettazione del sistema è stata guidata dai seguenti obiettivi:

- separare chiaramente frontend, backend e persistenza;
- mantenere il backend indipendente dalla tecnologia utilizzata per la interfaccia utente;
- esporre le funzionalità tramite API REST;
- centralizzare la logica applicativa nel backend;
- distinguere controller, regole di dominio e persistenza tramite Entity Framework Core;
- limitare la complessità, mantenendo il sistema coerente con il lavoro di un gruppo di tre studenti;
- rendere il progetto facilmente testabile, estendibile e spiegabile in sede di discussione.

3 Architettura generale

Il sistema è stato progettato come applicazione web distribuita composta da tre macro componenti:

- **Frontend Angular**, responsabile della interfaccia utente e della interazione con lo utente;
- **Backend ASP.NET Core Web API**, responsabile della logica applicativa, della autenticazione, della gestione dei ruoli e della esposizione delle API REST;
- **Database PostgreSQL**, responsabile della persistenza dei dati.

La comunicazione tra frontend e backend avviene esclusivamente tramite richieste HTTP verso API REST. Le richieste protette includono un token JWT generato dal backend durante la fase di login.

Questa scelta permette di rispettare il vincolo di separazione tra frontend e backend: il backend non dipende da Angular e potrebbe essere utilizzato anche da una diversa interfaccia, per esempio una applicazione mobile o desktop.

3.1 Component diagram

Il frontend contiene componenti e servizi Angular che invocano le API. Nel backend i controller usano `ApplicationDbContext`; `IssueDomainService` contiene regole di dominio e i servizi JWT e hashing gestiscono l'autenticazione. Non sono presenti repository applicativi.

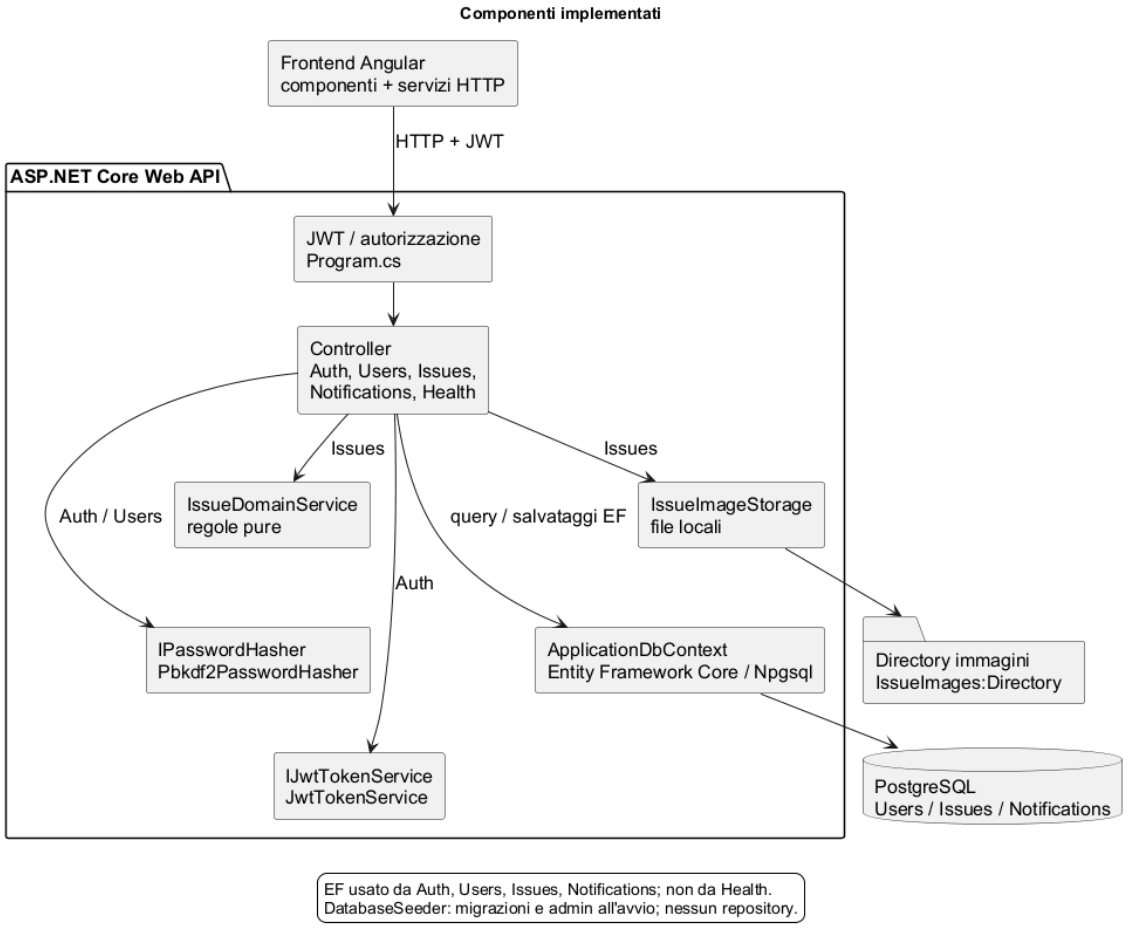


Figura 1: Componenti effettivi: controller, regole pure e persistenza EF

Il PNG è rigenerato da `diagrammi_architetturali/component_diagram.puml`. Il sorgente PlantUML sostituisce il relativo `.vpd`, conservato soltanto come storico. Sono rappresentati anche i servizi di hashing/JWT e `IssueImageStorage`, già presente nel checkout; la sua presenza non equivale al collaudo del flusso immagini in Docker.

3.2 Deployment diagram

Il deployment diagram mostra una possibile distribuzione del sistema. Lo utente accede alla applicazione tramite browser. La parte frontend comunica con il backend tramite rete, mentre il backend accede al database PostgreSQL tramite protocollo PostgreSQL.

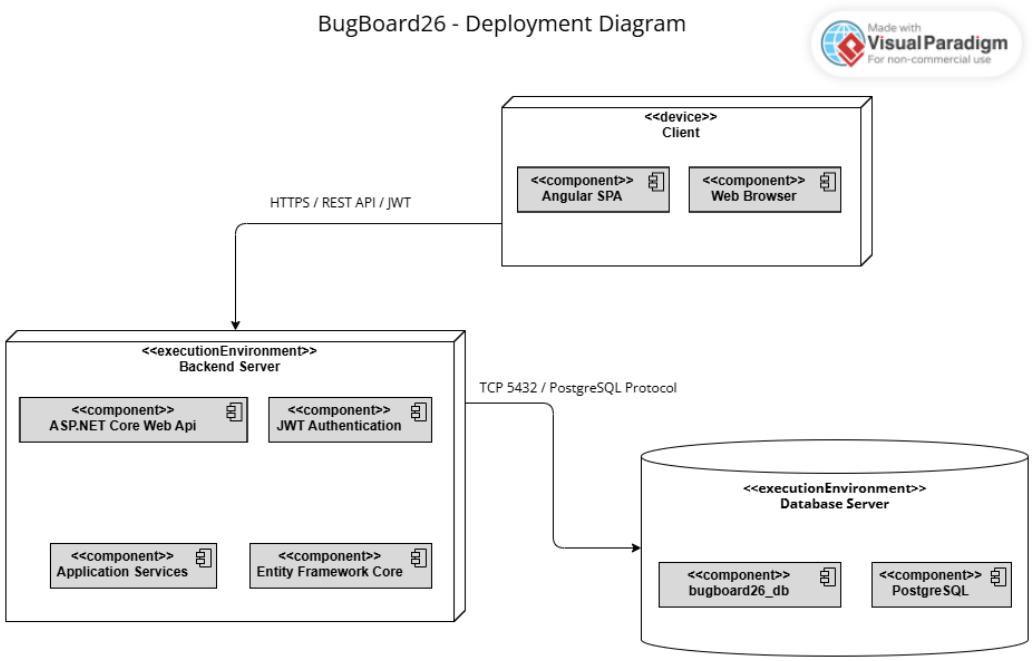


Figura 2: Deployment diagram di BugBoard26

Nell'avvio locale attuale Compose esegue PostgreSQL 16 e il backend .NET 10. Il backend usa `postgres:5432` e pubblica la porta interna 8080 su `localhost:5256`; Angular resta sul PC, sulla porta 4200. I volumi nominati conservano database e file immagine; il secondo è montato in `/var/lib/bugboard26/issue-images`. In alternativa l'API può essere avviata con `dotnet run`, fermando prima il backend Docker. Il deployment diagram originale resta una vista logica: i dettagli locali sono in `docker/docker-compose.yml` e nel README principale.

4 Scelte tecnologiche

4.1 Frontend Angular

Angular è stato scelto per realizzare una Single Page Application strutturata in componenti. Questa tecnologia permette di separare la presentazione dalla logica di accesso alle API e offre strumenti già integrati per routing, form, servizi e gestione delle chiamate HTTP.

Nel frontend sono presenti componenti dedicati alle principali schermate:

- login;
- lista issue;
- creazione issue;
- dettaglio issue;
- archivio issue;
- notifiche;
- gestione utenti per amministratori.

4.2 Backend ASP.NET Core Web API

ASP.NET Core Web API è stato scelto per il backend perché supporta in modo nativo la creazione di API REST, la gestione della dependency injection, la autenticazione JWT e la integrazione con Entity Framework Core.

La struttura implementata distingue:

- **Controller**: validano richieste, applicano autorizzazioni, interrogano il contesto e restituiscono DTO;
- **IssueDomainService**: verifica i permessi di cambio stato e calcola il suggerimento dell'assegnatario;
- **ApplicationDbContext**: mappa entità e relazioni e salva le modifiche tramite Entity Framework Core;
- **Servizi di autenticazione**: `Pbkdf2PasswordHasher` e `JwtTokenService`, con le rispettive interfacce;
- **IssueImageStorage**: validazione e memorizzazione dei file immagine, separati dalle righe PostgreSQL.

Questa struttura consente di separare le responsabilità e rende più semplice testare la logica applicativa.

4.3 Database PostgreSQL

PostgreSQL è stato scelto come database relazionale per la persistenza delle informazioni principali del sistema. Le entità principali sono utenti, issue e notifiche.

La scelta di un database relazionale è coerente con la natura dei dati, che presentano relazioni chiare tra utenti, issue e notifiche.

4.4 Autenticazione JWT

La autenticazione è basata su token JWT. Dopo il login, il backend restituisce un token contenente le informazioni essenziali dello utente, tra cui identificativo e ruolo. Il frontend invia il token nelle richieste successive verso le API protette.

Questa soluzione permette di mantenere il backend stateless rispetto alle sessioni e di gestire in modo semplice autorizzazioni basate sui ruoli.

5 Organizzazione del backend

Il backend è stato progettato separando le responsabilità in più livelli.

5.1 Controller

I controller rappresentano il punto di ingresso delle richieste HTTP. Ogni controller espone un gruppo di API legate a una specifica area funzionale:

- `AuthController`: login e generazione del token JWT;
- `UsersController`: creazione e consultazione degli utenti;
- `IssuesController`: creazione, ricerca, filtro, cambio stato, archiviazione, duplicati, export, suggerimento assegnatario e API immagini;
- `NotificationsController`: consultazione e lettura delle notifiche.

5.2 Servizi applicativi

`IssueDomainService` lavora su oggetti del dominio, senza dipendere dal database. Espone `CanChangeIssueStatus`, `SuggestAssignee` e `SearchIssues`. La ricerca API usa invece `ILike` nelle query di `IssuesController`; l'export CSV è costruito nello stesso controller. Il controller aggiorna stato e date, aggiunge la notifica e chiama `SaveChangesAsync`. Non esistono `IssueService`, `NotificationService` o repository nel codice attuale.

5.3 DbContext

`ApplicationDbContext` rappresenta il punto di integrazione con Entity Framework Core e PostgreSQL. Configura indici, lunghezze dei campi, conversioni degli enum e relazioni. Espone i `DbSet`:

- `Users`;
- `Issues`;
- `Notifications`.

`Program.cs` chiama `DatabaseSeeder.SeedAdminAsync` prima di accettare richieste. Il seeder esegue `Database.MigrateAsync()` e crea un admin attivo solo se manca un utente di ruolo ADMIN. Le migrazioni presenti sono in `backend/BugBoard26.Api/Migrations`; non serve una seconda procedura manuale per applicarle. Connessione, JWT e credenziali iniziali sono configurabili tramite variabili d'ambiente.

6 Class diagram di design

Il sorgente `diagrammi_dettaglio/class_diagram.puml` genera tre PNG: due viste di dettaglio per dominio/persistenza e controller/dipendenze, più la vista d'insieme `class_diagram.png`. Sostituisce il `class_diagram.vpd` storico; non introduce repository o servizi assenti dal codice.

6.1 Vista dominio e persistenza

La vista dominio e persistenza mostra le entità principali del sistema e le relazioni tra esse.

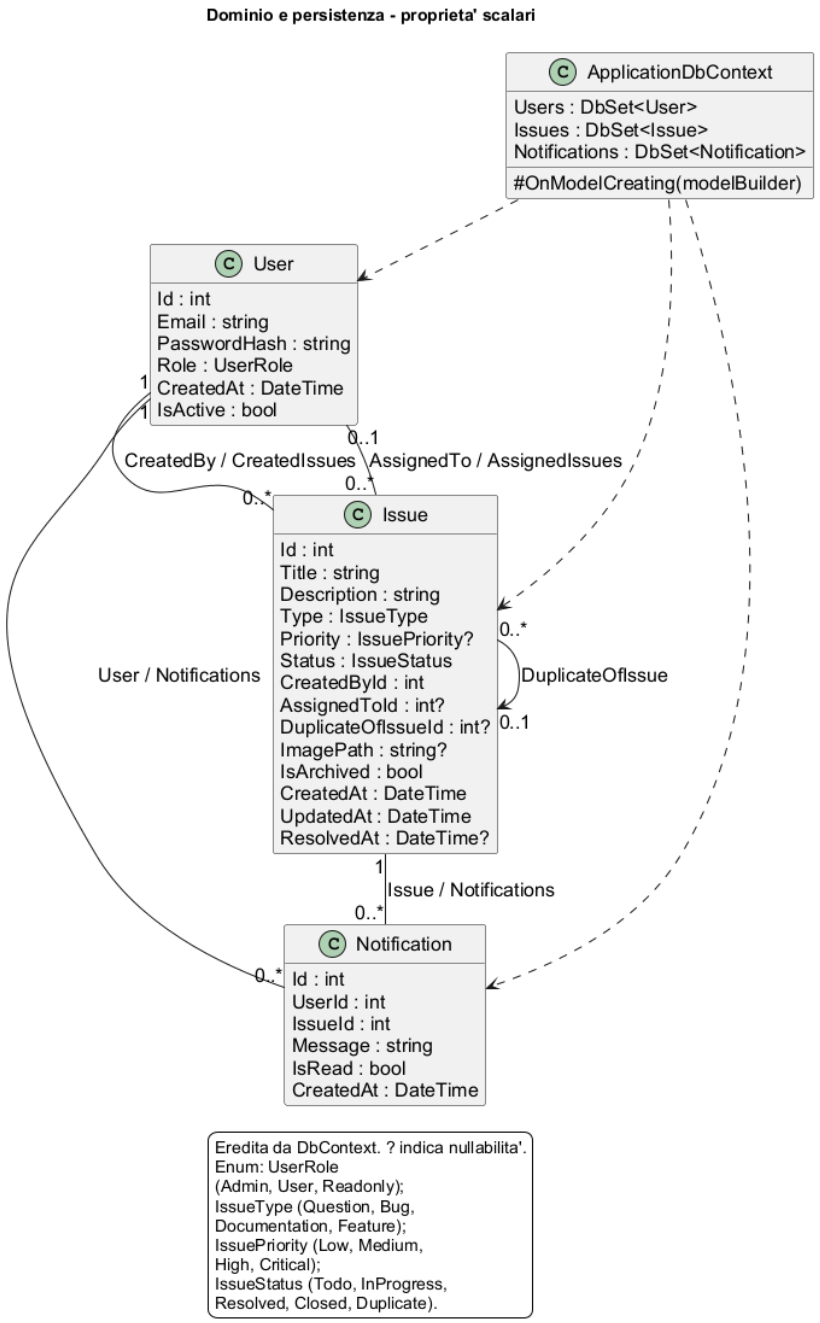


Figura 3: Class diagram: dominio e ApplicationDbContext

6.2 Vista controller e dipendenze

La seconda vista mostra i controller e le dipendenze ricevute tramite injection. Le operazioni sono elencate per nome, con firme abbreviate per leggibilità; DTO e dipendenze di framework sono indicati nella legenda, senza riprodurre l'intera API di ASP.NET Core.

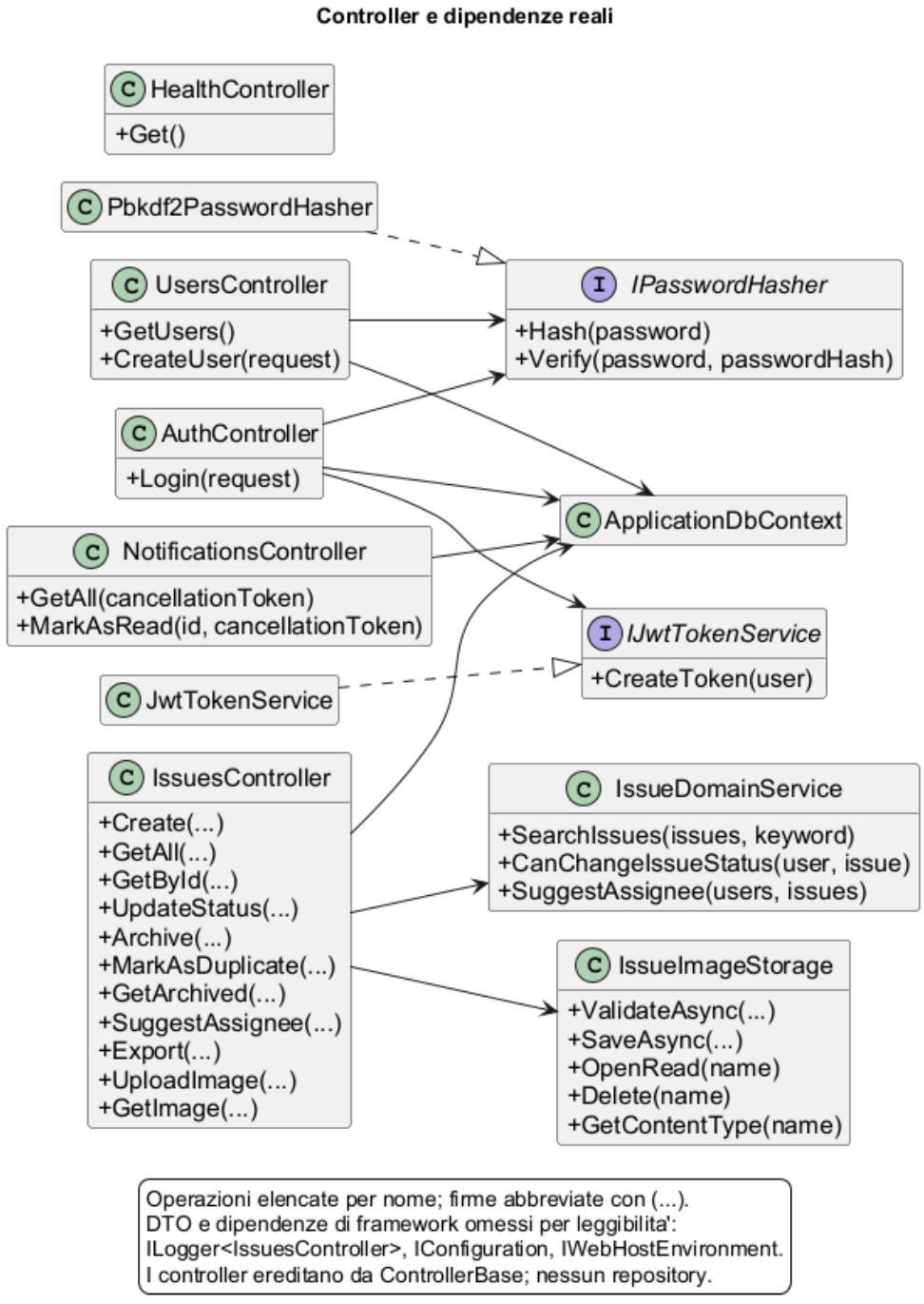


Figura 4: Class diagram: controller e servizi implementati

`ApplicationDbContext` eredita da `DbContext`; i controller da `ControllerBase`. `Pbkdf2PasswordHasher` e `JwtTokenService` implementano le due interfacce di autenticazione. `IssueDomainService` non accede al database. `HealthController` non riceve il contesto: il suo endpoint non prova la disponibilità di PostgreSQL.

7 Schema della persistenza dati

La persistenza dei dati è basata su tre tabelle principali:

- `Users`;
- `Issues`;
- `Notifications`.

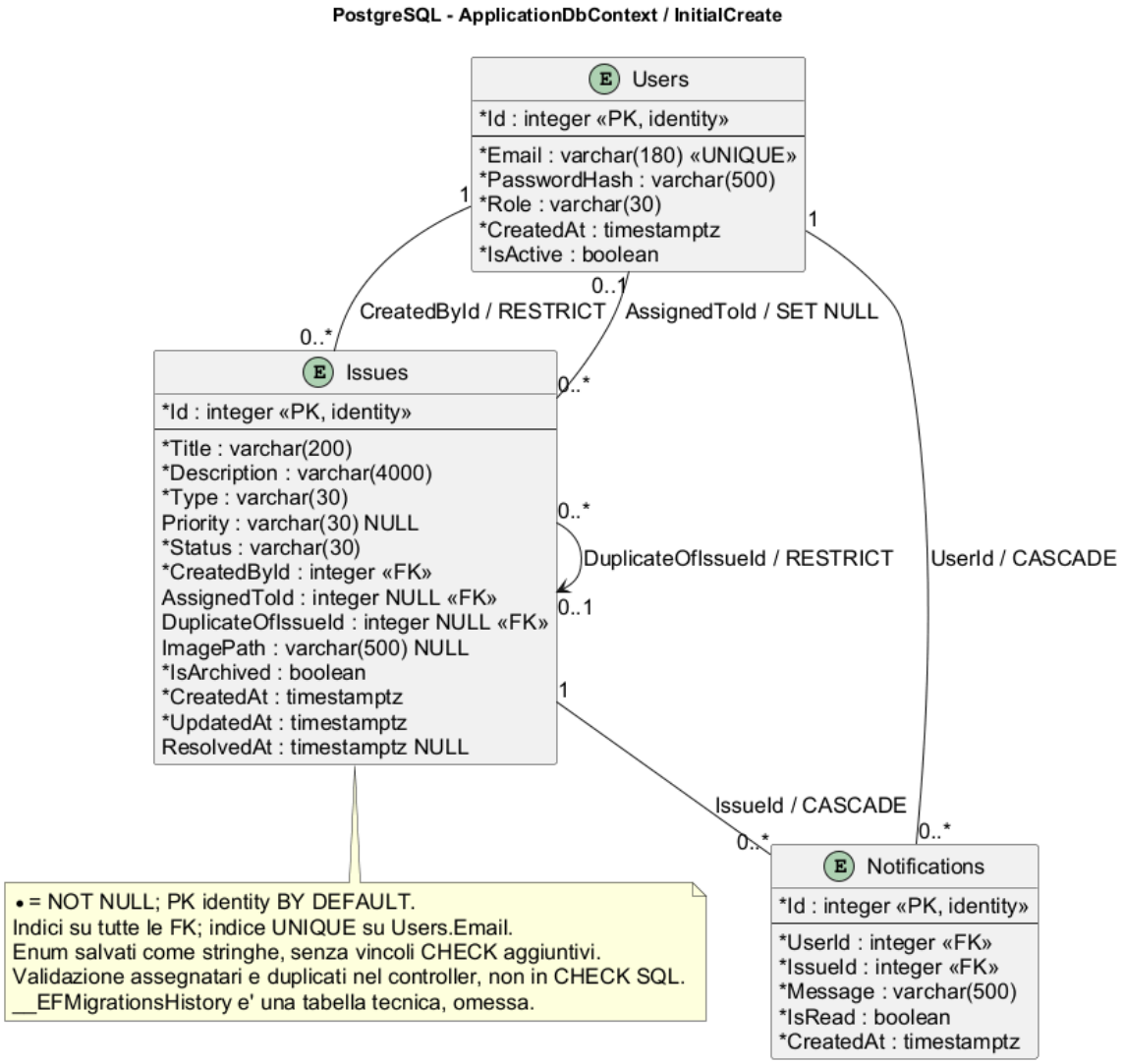


Figura 5: Schema PostgreSQL effettivo: tipi, nullabilità e chiavi esterne

Tabella	Descrizione
`Users`	Contiene gli utenti, l'hash della password, il ruolo e lo stato di attivazione. L'hashing non è cifratura reversibile.
`Issues`	Contiene le issue create dagli utenti, il loro stato, la priorità, il tipo, lo assegnatario, le informazioni di archiviazione e lo eventuale collegamento a una issue duplicata.
`Notifications`	Contiene le notifiche generate dal sistema, in particolare quelle relative alla risoluzione di una issue.

Il PNG è generato da `diagrammi_dettaglio/database_schema.puml`, sostituto del `.vpd` storico, confrontato con `ApplicationDbContext` e la migrazione `InitialCreate`. Le PK sono `integer` con identity; le date sono `timestamp with time zone`. L'email ha indice univoco; tutte le FK hanno un indice. La cancellazione del segnalatore o dell'originale duplicato è `Restrict`, quella dell'assegnatario è `SetNull`; le notifiche usano `Cascade` per entrambe le FK. Gli enum sono stringhe, senza vincoli CHECK aggiuntivi. La validazione di account attivi, ruoli e autoreferenza è nel controller, non nello schema. La tabella tecnica `__EFMigrationsHistory` è omessa dal diagramma.

8 Principali scelte di design

8.1 Separazione frontend e backend

Il frontend non accede direttamente al database e non contiene logica di business critica. Tutte le regole principali, come controllo dei ruoli, cambio stato, archiviazione e gestione dei duplicati, sono implementate nel backend.

8.2 Uso di servizi applicativi

Le regole pure di `IssueDomainService` sono verificabili senza database. Le operazioni persistenti restano nei controller e sono verificate con richieste HTTP e PostgreSQL nei test di integrazione.

8.3 Accesso ai dati

I controller ricevono `ApplicationDbContext` tramite dependency injection. Le query di consultazione usano `AsNoTracking`; le operazioni di modifica lavorano sulle entità tracciate e chiamano `SaveChangesAsync`. Non viene introdotto un livello repository aggiuntivo.

8.4 Gestione dei ruoli

Il sistema prevede tre ruoli principali:

- `ADMIN`: può gestire utenti, creare issue, cambiare stato, archiviare e contrassegnare duplicati;
- `USER`: può creare issue, consultarle e cambiare lo stato delle issue assegnate;
- `READONLY`: può consultare le issue, senza crearle o modificarle.

Il controllo dei ruoli viene effettuato lato backend, in modo da impedire operazioni non autorizzate anche in caso di manipolazione della interfaccia frontend.

Lista, dettaglio, archivio, ricerca, export CSV e suggerimento assegnatario sono accessibili a tutti e tre i ruoli autenticati. Nel frontend `canChangeStatus` mostra il pannello solo ad ADMIN o USER assegnatario; `canCreateIssue` e `issueCreationGuard` escludono READONLY dalla creazione, mentre `adminGuard` protegge la pagina utenti. Riferimenti: `frontend/src/app/pages/issue-detail-page/issue-detail-page.component.ts`, `frontend/src/app/services/auth.guard.ts` e `frontend/src/app/app.routes.ts`.

Requisiti del cambio stato. RF-10 e UC-02 consentono l'operazione all'assegnatario oppure all'amministratore; RF-17 esclude readonly dalle modifiche alle issue. RNF-02 richiede il controllo sul backend e RNF-05 richiede che l'interfaccia renda chiare le operazioni disponibili per ruolo.

Comportamento attuale. `IssuesController.UpdateStatus` espone PATCH `/api/issues/{id}/status` per `ADMIN`, `USER`. Verifica l'account attivo e chiama `IssueDomainService.CanChangeIssueStatus`: admin può agire su qualsiasi issue, USER deve essere assegnatario e READONLY è escluso anche se assegnato. Una richiesta non autorizzata è rifiutata con 403; l'account corrente assente o inattivo produce 401, una issue inesistente 404.

Duplicati e transizioni.

- Il frontend propone `TODO`, `IN_PROGRESS`, `RESOLVED`, `CLOSED` e disabilita il cambio per una issue `DUPLICATE`. Non esiste una matrice generale di transizioni: non va inventata una sequenza obbligatoria.
- Nel codice ora presente `UpdateStatus` rifiuta il nuovo stato `Duplicate` con 400, anche per ADMIN; richiede il percorso dedicato con originale. Se la issue è già `Duplicate`, oppure ha `DuplicateOfIssueId`, restituisce 409. Questi controlli precedono anche la richiesta dello stesso stato. Le precedenti annotazioni sul merge delle correzioni non descrivono più il checkout attuale.

Riferimenti: `backend/BugBoard26.Api/Controllers/IssuesController.cs`, metodi `UpdateStatus` e `MarkAsDuplicate`; `backend/BugBoard26.Api/Services/IssueDomainService.cs`, metodo `CanChangeIssueStatus`.

8.5 Suggerimento automatico dello assegnatario

`IssueDomainService.SuggestAssignee` considera gli utenti attivi non READONLY, inclusi gli ADMIN, e conta le issue non archiviate in stato `TODO` o `IN_PROGRESS`. Sceglie il carico minore e, a parità, l'identificativo utente minore. Questo descrive il criterio implementato, senza assumere un ordinamento per email.

In `IssuesController.Create`, `AssignedToId` può essere nullo. Se valorizzato, deve identificare un utente esistente, attivo e non READONLY: ADMIN e USER attivi sono validi. In caso contrario l'API restituisce 400 `ValidationProblem` senza creare la issue. Il suggerimento non sostituisce questo controllo sul backend.

GET `/api/issues/suggest-assignee` restituisce ID, email e carico; senza candidati risponde 404. Non modifica né assegna automaticamente una issue e non genera una notifica di assegnazione.

8.6 Gestione delle notifiche

Requisito. RF-11 richiede la notifica al segnalatore quando la issue passa a `RESOLVED`. Per la verifica di questo flusso, ogni utente deve consultare e segnare come lette soltanto le proprie notifiche.

Comportamento attuale della risoluzione. In `IssuesController.UpdateStatus`, se lo stato richiesto differisce dal corrente, vengono aggiornati `Status` e `UpdatedAt`. Per `RESOLVED` viene impostato `ResolvedAt` in UTC e aggiunta una `Notification` con `UserId = CreatedById`, `IssueId`, messaggio di risoluzione, `IsRead = false` e `CreatedAt` in UTC. Una sola chiamata `SaveChangesAsync` persiste issue e notifica.

Per una issue non duplicata, la ripetizione dello stato corrente valido risponde 200 senza scritture, date nuove o ulteriori notifiche. Uscendo da `RESOLVED`, `ResolvedAt` viene azzerato senza eliminare o cambiare la lettura delle notifiche precedenti. Un nuovo ingresso in `RESOLVED` imposta una nuova data e crea una nuova notifica. Non è presente un invio email o un aggiornamento in tempo reale della pagina notifiche.

Comportamento attuale delle notifiche. `NotificationsController.GetAll` restituisce con GET `/api/notifications` solo le notifiche con `UserId` uguale all'utente corrente, dalla più recente. `MarkAsRead`, PATCH `/api/notifications/{id}/read`, verifica il proprietario prima di impostare `IsRead = true`. Restituisce 403 per la notifica altrui, 404 per un ID inesistente e 401 per account corrente assente o inattivo. Anche ADMIN può operare solo sulle proprie notifiche. Ripetere la lettura di una propria notifica restituisce 200 e mantiene `IsRead = true`; non esiste un endpoint di dettaglio notifica per ID.

La pagina Angular carica l'elenco all'apertura, evidenzia le notifiche non lette e aggiorna l'elemento con la risposta del PATCH. Il pulsante *Segna come letta* è mostrato solo per notifiche non lette. Il dettaglio issue visualizza `ResolvedAt`; `IssuesService` converte i nomi di stato Angular nei nomi API, ad esempio `RESOLVED` in `Resolved`.

Il controller notifiche non usa l'attributo `Authorize`: entrambi i metodi controllano il claim `sub` e l'account attivo tramite `GetCurrentUserAsync`. Questo controllo interno produce 401 in assenza di un utente valido; non si descrive un attributo assente dal codice.

Riferimenti: `backend/BugBoard26.Api/Controllers/NotificationsController.cs`; modelli `backend/BugBoard26.Api/Models/Issue.cs` e `backend/BugBoard26.Api/Models/Notification.cs`; pagine `frontend/src/app/pages/notifications-page/notifications-page.component.ts` e relativo HTML; `frontend/src/app/services/notifications.service.ts`.

Differenza da chiarire. RF-17 parla di readonly *senza modificare dati*, mentre il design circoscrive il divieto alle issue. `NotificationsController.MarkAsRead` e la pagina notifiche non escludono READONLY: un account attivo di questo ruolo può segnare come letta una propria notifica. Se RF-17 vieta anche questa scrittura, il codice non è conforme; se il divieto riguarda solo le issue, il requisito va chiarito con il committente. RF-17 resta invariato e non si assume una nuova eccezione.

8.7 Gestione dei duplicati

UC-03 riserva il contrassegno all'amministratore e richiede la issue originale. `MarkAsDuplicate` imposta `Status` a `Duplicate`, collega `DuplicateOfIssueId`, azzera `ResolvedAt` e aggiorna `UpdatedAt` in UTC. Rifiuta l'autoreferenza con 400, restituisce 404 se manca la issue da contrassegnare e 400 se manca l'originale. Salva una volta, senza nuova notifica di risoluzione. Le notifiche precedenti restano presenti. Non sono implementati controlli ulteriori su catene o cicli di duplicati. L'autorizzazione del metodo usa il ruolo JWT ADMIN, senza il lookup dell'account attivo usato da `UpdateStatus`.

8.8 Archiviazione

Solo gli amministratori possono archiviare una issue. Una issue archiviata non viene mostrata nella lista principale, ma resta accessibile da una sezione dedicata. Questa scelta permette di non eliminare dati utili e di conservare la tracciabilità delle segnalazioni.

8.9 Consultazione ed export CSV

Lista ed export condividono la query di `IssuesController`: escludono gli archiviati, filtrano per `Type`, `Status`, `Priority`, `CreatedById`, `AssignedToId` e `Keyword`. La ricerca usa `ILike` su titolo e descrizione; l'ordinamento usa data, priorità, stato o titolo, ascendente o discendente. L'archivio usa invece una query separata, ordinata per `UpdatedAt` decrescente.

L'export UTF-8 contiene ID, titolo, tipo, priorità, stato, email di segnalatore/assegnatario e data di creazione. Gestisce virgole, virgolette e ritorni a capo. Non comprende immagini, descrizione o notifiche e non è un backup completo del database.

8.10 Immagine opzionale della issue

POST `/api/issues/{id}/image` carica o sostituisce una immagine: `multipart/form-data`, campo `image`, per ADMIN o USER segnalatore, non il solo assegnatario. READONLY è escluso. GET consente lettura autenticata anche degli archiviati; il DTO espone `HasImage`, non il percorso fisico.

`IssueImageStorage` ammette PNG/JPEG fino a 5 MiB, controlla estensione e marcatori, non decodifica completa o antivirus. Salva con nome GUID fuori da `wwwroot`; PostgreSQL conserva `ImagePath`. Il controller usa transazione e lock sulla issue per la sostituzione, rimuove il vecchio file e pulisce quello nuovo se il salvataggio fallisce. Database e filesystem non formano una transazione distribuita.

Docker usa un volume immagini, scrivibile dall'utente non root. L'avvio sul PC salva nella cartella `uploads/issue-images` dell'API, senza condividere i file del container. Collaudo end-to-end e persistenza restano da confermare con i colleghi.

9 Diagrammi comportamentali

Le sequenze sono rigenerate dai rispettivi sorgenti `.puml` nella cartella `diagrammi_comportamentali`; i `.vpd` originali restano storici e non vanno usati per riesportare questi PNG. Creazione e duplicazione mostrano query e salvataggio del controller sul contesto,

senza repository o servizi intermedi. Il flusso del cambio stato è mantenuto, con i soli controlli 400/409 sui duplicati aggiunti e la nota obsoleta sul merge rimossa.

9.1 Creazione di una issue

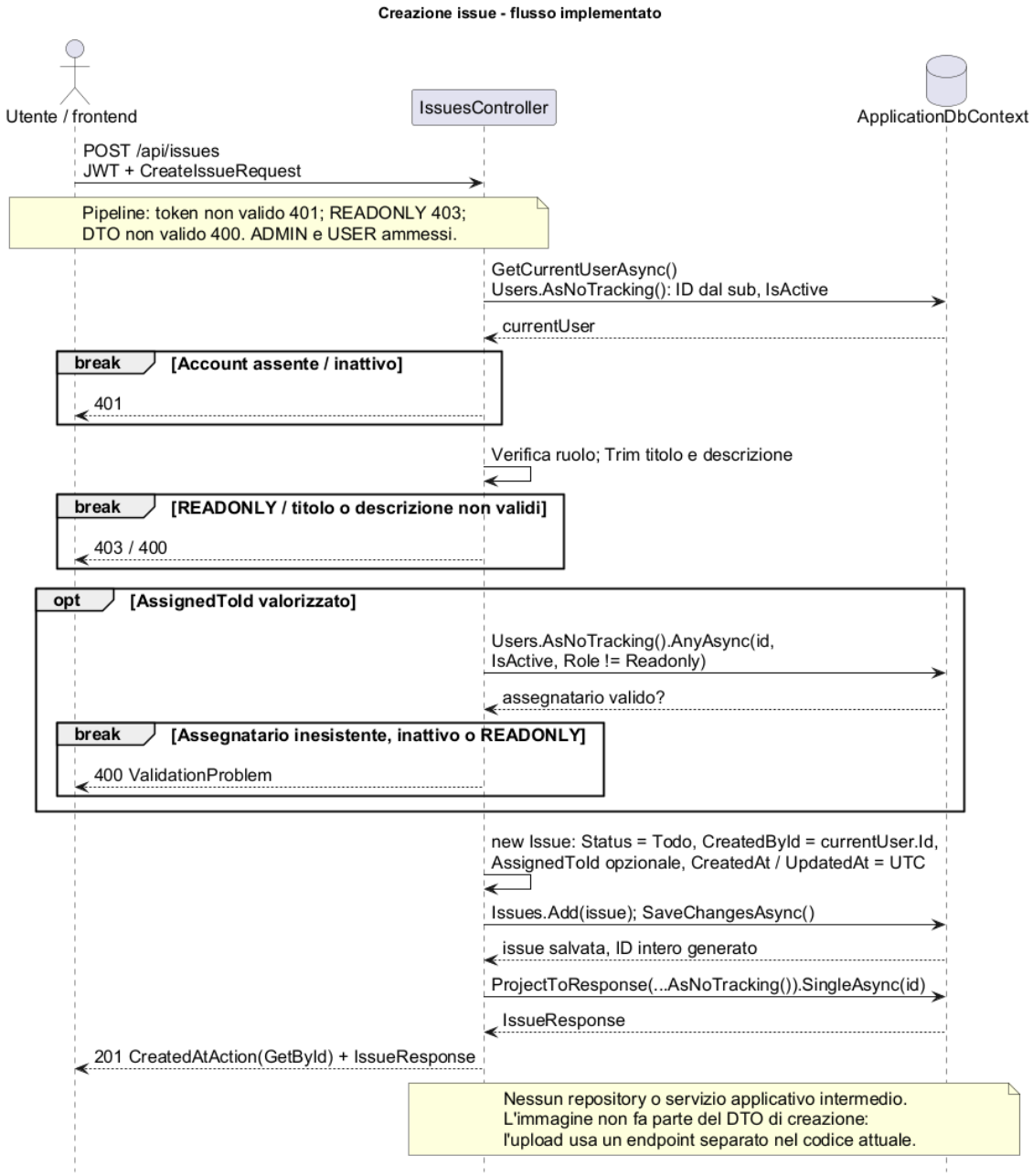


Figura 6: Sequence diagram: creazione di una issue

9.2 Cambio stato con notifica

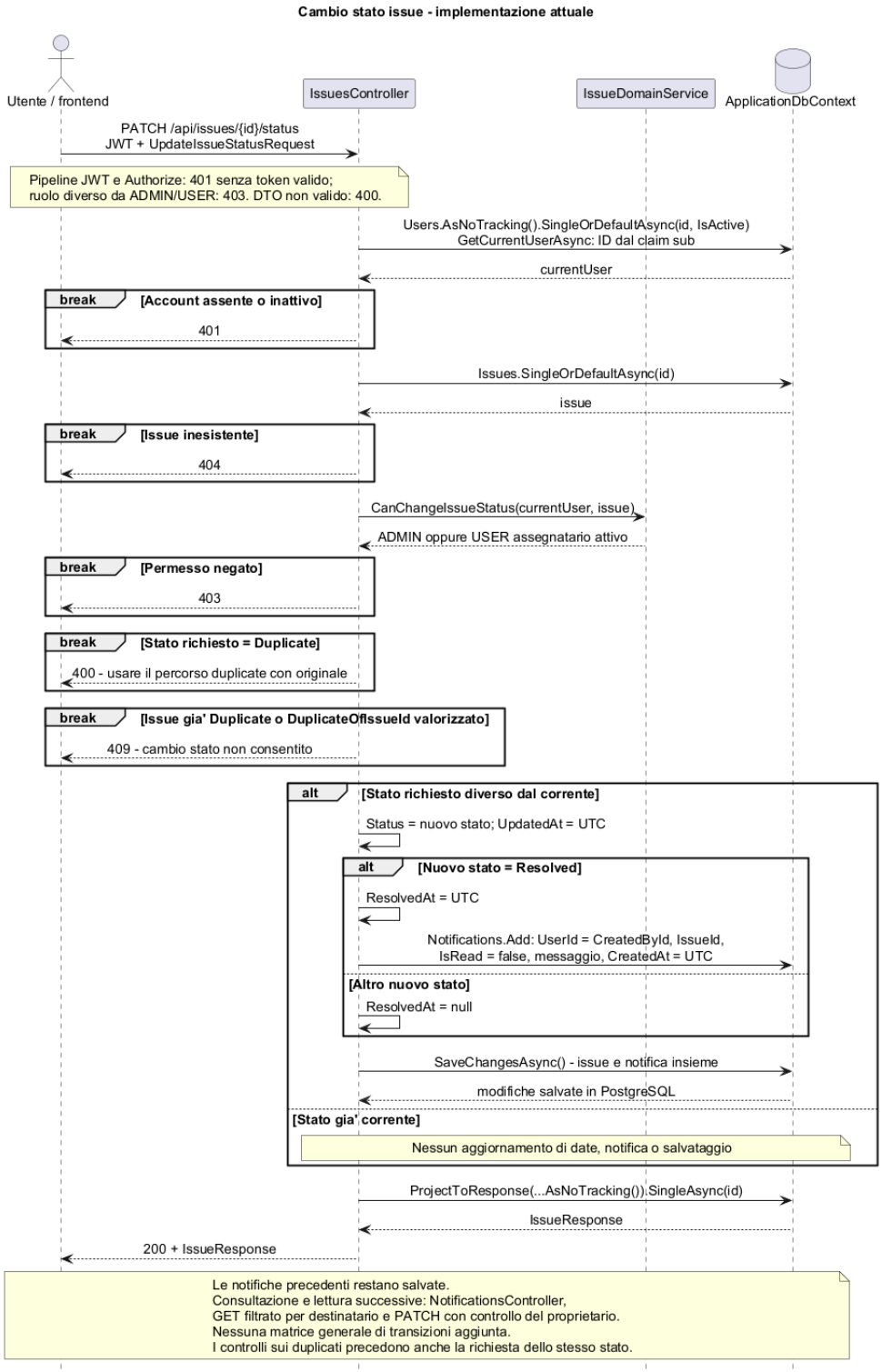


Figura 7: Cambio stato con notifica: flusso implementato

Confronto con UC-02 e con il codice. Il sorgente aggiornato è `sequence_cambio_stato.puml`. Sono mantenuti autorizzazione, account attivo, caricamento della issue, `CanChangeIssueStatus`, date, eventuale notifica e un solo `SaveChangesAsync`. I controlli sui duplicati precedono il confronto con lo stato corrente: il ramo senza scritture riguarda quindi solo stati validi di issue non duplicate. Consultazione e lettura della notifica restano operazioni successive di `NotificationsController`. Non viene introdotta una matrice generale di transizioni.

9.3 Contrassegno di una issue duplicata

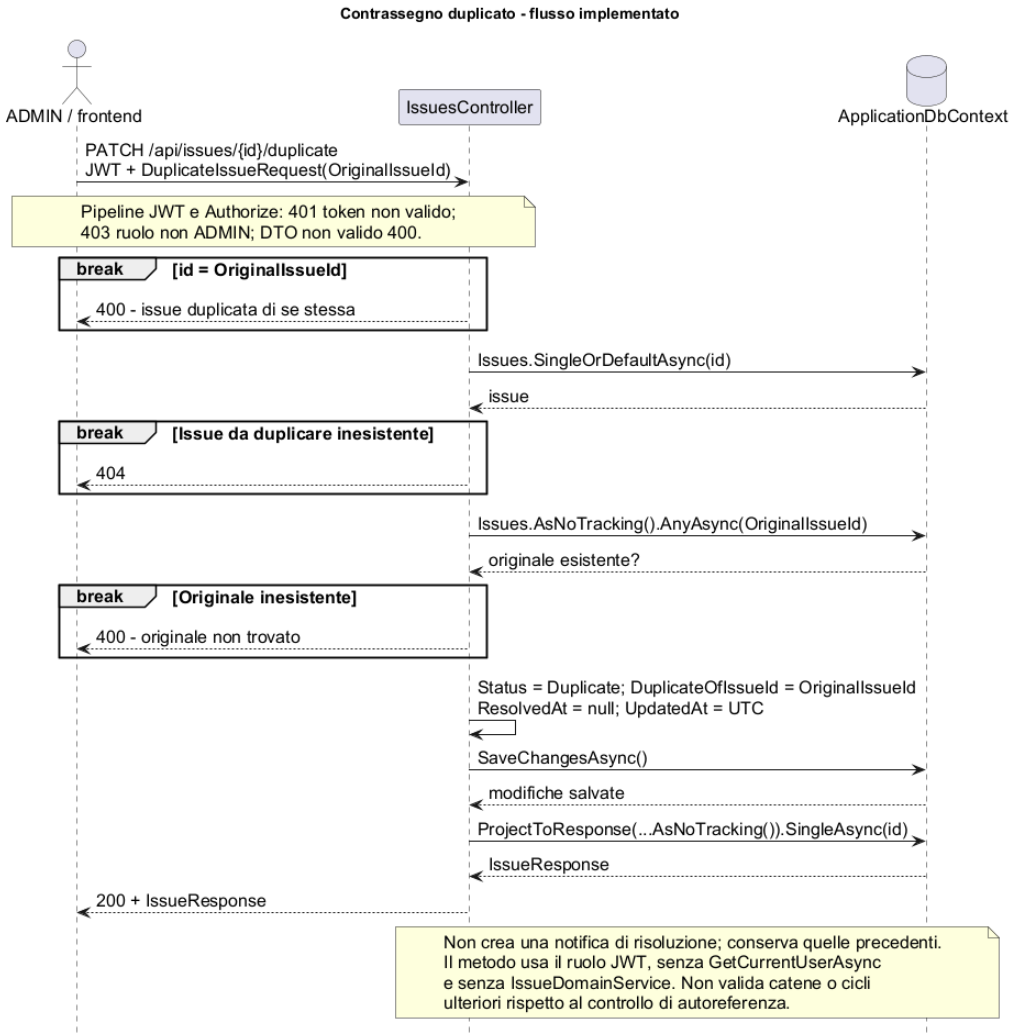


Figura 8: Sequence diagram: contrassegno di una issue duplicata

10 API principali implementate

Metodo	Endpoint	Descrizione
POST	/api/auth/login	Effettua il login e restituisce il token JWT.
POST	/api/users	Crea un nuovo utente. Operazione riservata agli amministratori.
GET	/api/users	Restituisce la lista degli utenti; solo ADMIN.
POST	/api/issues	Crea una nuova issue; ADMIN o USER.
GET	/api/issues	Restituisce le issue con ricerca, filtri e ordinamento.
GET	/api/issues/{id}	Restituisce il dettaglio di una issue.
PATCH	/api/issues/{id}/status	Modifica lo stato per ADMIN o USER assegnatario; READONLY escluso.
PATCH	/api/issues/{id}/archive	Archivia una issue. Operazione riservata agli amministratori.
GET	/api/issues/archived	Restituisce le issue archiviate.
PATCH	/api/issues/{id}/duplicate	Contrassegna una issue come duplicata; solo ADMIN.
GET	/api/issues/export	Esporta le issue in formato CSV.
GET	/api/issues/suggest-assignee	Suggerisce un utente attivo non READONLY; accessibile a tutti i ruoli.
POST	/api/issues/{id}/image	Carica o sostituisce l'immagine; ADMIN o USER segnalatore.
GET	/api/issues/{id}/image	Legge l'immagine con autenticazione; 404 se assente.
GET	/api/notifications	Restituisce le notifiche dell'utente autenticato.
PATCH	/api/notifications/{id}/read	Segna una propria notifica come letta; 403 se appartiene ad altri.

Tabella 2: Endpoint principali esposti dal backend

11 Testabilità

`backend/BugBoard26.Tests/IssueDomainServiceTests.cs` verifica regole pure di ricerca, carico e autorizzazione. I test di integrazione avviano `Program` tramite `WebApplicationFactory`, usano JWT ottenuti con il login reale, Npgsql e un PostgreSQL dedicato sulla porta 55432. Ogni caso crea un database temporaneo distinto; non usa il database di sviluppo né repository simulati.

`IssueStatusIntegrationTests` verifica risoluzione da parte dell'assegnatario con segnalatore distinto, date e notifica persistite, ADMIN non assegnatario, rifiuto di USER non assegnatario e READONLY, ripetizione di `Resolved` senza nuove date/notifiche, 401 senza token e 404 per issue assente. `NotificationsIntegrationTests` verifica elenco personale ordinato, lettura persistita per USER/ADMIN/READONLY, rifiuto di notifiche altrui, 401, 404 ed elenco vuoto. Il successo di questi test descrive il comportamento attuale, non chiarisce da solo RF-17.

`IssueDuplicateValidationIntegrationTests` verifica 400 per `Duplicate` via status, 409 per issue già duplicate (anche marcatori legacy), azzeramento di `ResolvedAt` e assenza di nuove notifiche nel percorso dedicato. `IssueAssigneeValidationIntegrationTests` verifica assegnazione nulla, ADMIN/USER attivi validi e rifiuto di account inesistenti, inattivi o READONLY. Sono presenti anche test di immagini, export CSV e suggerimento assegnatario.

La riapertura dopo `RESOLVED`, una seconda risoluzione e la ripetizione del PATCH read non hanno casi dedicati nei due file di stato/notifiche: restano nella checklist manuale. Esiti e limiti sono registrati in `Documentazione/Documentazionefinale/readme.md`; la compilazione LaTeX non è una prova applicativa. La prova locale preliminare del 9 ottobre sul commit `15ca2e2` ha eseguito solo i test di dominio con coverage parziale. La suite PostgreSQL e le verifiche dei colleghi non sono dichiarate concluse; non si fissa un conteggio finale di test o copertura.

11.1 Analisi qualità backend

È predisposta una sola soluzione: SonarQube Server locale, senza pubblicazione cloud. `quality/coverage.runsettings` riusa `coverlet.collector` già presente e genera OpenCover per l'assembly API, escludendo migrazioni, `bin/obj` e sorgenti generati, non controller o altro codice applicativo. `quality/analyze-backend.ps1` esegue build Release e test; la modalità Analysis aggiunge `begin/end` e importa il report soltanto dopo il successo dei test. Richiede server locale, progetto già esistente e token nella sessione, senza crearli. Istruzioni ed esiti sono in `Documentazione/Documentazionefinale/qualita_backend.md`. L'analisi Sonar e il Quality Gate non sono stati eseguiti.

12 Limiti e verifiche aperte

- Nel checkout sono presenti endpoint immagini, `IssueImageStorage` e test dedicati. Il collaudo frontend/API e la persistenza dei file con ricreazione del container restano da verificare sul setup condiviso; non si dichiara completata la consegna immagini dei colleghi sulla sola lettura del codice.
- RF-17 da chiarire sulla lettura delle proprie notifiche per READONLY; nessuna eccezione viene aggiunta al requisito.

- I `.vpd` conservano il disegno storico; per le viste aggiornate usare i sorgenti PlantUML e PNG corrispondenti. Il deployment originale resta una vista logica, non rigenerata in questa attività.
- Suite di integrazione con coverage e analisi SonarQube da eseguire quando database/server saranno disponibili. Resta l'avviso NU1903 sulla dipendenza `Microsoft.OpenApi` 2.0.0, non aggiornata in questa attività.
- Notifiche senza email o aggiornamento realtime.

13 Conclusioni

La progettazione proposta divide BugBoard26 in componenti indipendenti e con responsabilità chiare. Il frontend Angular si occupa della presentazione, il backend ASP.NET Core contiene la logica applicativa e PostgreSQL gestisce la persistenza dei dati.

La struttura a livelli adottata rende il sistema più ordinato, testabile ed estendibile. Le scelte di design sono coerenti con le funzionalità assegnate e con il vincolo di realizzare un sistema distribuito con frontend e backend separati.